

Fincept

End-to-End Big Data Financial Pipeline

Medallion Architecture | Real-time Processing | Containerized

Apache Spark | Kafka | Cassandra | PostgreSQL | Docker

Fincept: End-to-End Big Data Financial Pipeline

Fincept is a high-performance, containerized big data pipeline designed to ingest, process, and visualize real-time financial market data. It implements a **Medallion Architecture** to ensure data quality and scalability, transforming raw API data into production-ready financial insights.

Concepts for Beginners (Intern Guide)

If you are new to Big Data, here are the core concepts used in this project explained simply:

1. The "Assembly Line" (Pipeline)

Think of this project as a factory assembly line.

- * **Raw Materials**: Stock prices from Yahoo Finance.
- * **Stations**: Each container (Spark, Kafka, etc.) is a station that changes the data (cleaning it, adding calculations) until it becomes a finished product (a chart in the UI).

2. Distributed Computing (The Team Analogy)

Imagine you have to count 1,000,000 words. One person (a single computer) would take a long time.

- * **Spark** is like a manager. It breaks the 1,000,000 words into 10 piles and gives them to 10 workers (**Slaves/Workers**). They count their piles at the same time (parallel) and give the totals back to the manager. This makes the work 10x faster.

3. Medallion Architecture (The Filtering Process)

We don't just save data once. we "polish" it in stages:

- * **Bronze**: Raw, messy data straight from the internet.
- * **Silver (Cassandra)**: Cleaned data that is easy to read but still very detailed (minute-by-minute).
- * **Gold (Postgres)**: High-quality, aggregated data (daily totals) used for final reports.

4. Message Brokers (The Post Office)

Kafka is like a post office. The data feeder (Sender) drops off thousands of "letters" (prices). Kafka holds them safely. Spark (Receiver) picks them up when it's ready. If Spark is slow for a minute, the letters don't get lost they just wait at the post office.

5. Containers (The Shipping Analogy)

Docker allows us to put a whole computer system (Operating System + Software + Code) into a "box" (Container). This means the project will run exactly the same way on your laptop, a server, or a cloud VM without

any "it works on my machine" errors.

How It Works: The Workflow

The pipeline follows a structured data flow across several layers:

- * **Ingestion (Bronze)**:
 - The `yfinance_feeder.py` script polls the Yahoo Finance API for stock quotes.
 - Data is pushed into **Kafka** topics, serving as a resilient buffer.
- * **Streaming (Silver)**:
 - A **PySpark Streaming** job consumes data from Kafka in real-time.
 - Data is persisted into a **Cassandra Cluster**, which acts as the Silver layer (historical time-series data).
- * **Batch Processing (Gold)**:
 - Scheduled Spark jobs perform cleaning, deduplication, and aggregation.
 - **Aggregation Logic**: Minute data is rolled up into Hourly and Daily timeframes. Calculations include **OHLCV** (Open-High-Low-Close-Volume), Average Price, and **VWAP** (Volume Weighted Average Price).
 - **Feature Engineering**: The pipeline computes advanced technical indicators:
 - **Moving Averages**: 20, 50, and 200-period SMAs.
 - **Volatility**: Standard deviation of log returns.
 - **Bollinger Bands**: Upper/Lower bands based on 20-period moving averages and standard deviations.
 - Processed results are "promoted" to a **PostgreSQL Gold Layer**.
- * **Visualization**:
 - **Fincept Terminal**: A native C++ Qt application displays real-time analytics.
 - **Power BI**: Connects to the PostgreSQL Gold layer for executive dashboards.

Tech Stack

- * **Languages**: Python (Data logic), SQL/CQL (Storage), C++ (Desktop UI).
- * **Processing**: Apache Spark (v3.5.0) with PySpark.
- * **Messaging**: Apache Kafka (v7.5.0) with Zookeeper.
- * **Storage**:
 - **NoSQL**: Apache Cassandra (3-node cluster, v4.x).
 - **RDBMS**: PostgreSQL (v15-alpine).
- * **Orchestration**: Docker Compose & PowerShell.

Data Schemas (Conceptual)

1. Silver Layer (Cassandra)

- * **minute_data**: The rawest processed data. Columns: `symbol`, `timestamp`, `open`, `high`, `low`, `close`, `volume`.
- * **daily_data**: Aggregated OHLCV per day. Includes `vwap` and `avg\_price`.
- * **engineered_features**: Technical indicators linked to symbols and timestamps (SMA, Bollinger Bands, Volatility).

2. Gold Layer (PostgreSQL)

- * **daily_summary**: Highly refined table for Power BI. Optimized with indexes on `symbol` and `date`. Contains clean, audited financial summaries.

Container Breakdown

The project spawns 10 containers working in harmony:

Container	Role	Workflow / Responsibility
zookeeper	Coordinator	Manages Kafka broker state and leader elections.
kafka	Message Broker	Ingests raw data from feeders. Partitioned by `symbol` for parallel consumption.
cassandra-node[1-3]	Silver Storage	Distributed storage using a Replication Factor of 2. Uses `LeveledCompactionStrategy` for time-series optimization.
spark-master	Orchestrator	Manages the Spark cluster, schedules tasks, and provides the Web UI (Port 8080).
spark-worker[1-3]	Muscle	Executes Spark tasks. Each worker is allocated 4GB RAM and 2 Cores.
postgres-gold	Gold Storage	Relational database for final BI consumption. Stores structured daily summaries.
fincept-terminal	Frontend	A containerized UI that connects to Cassandra/Kafka. Uses X11 forwarding to appear on your host machine.

Infrastructure Details

Networking

All containers reside on an internal Docker bridge network named **financial-net**.

- * **Service Discovery**: Spark connects to Cassandra nodes by hostname (`cassandra-node1`, etc.).
- * **External Access**:
 - Postgres is mapped to `localhost:5432`.
 - Kafka is mapped to `localhost:9093` for host-side producers.

Reliability Features

- * **Health Checks**: Cassandra nodes have health checks that run `nodetool status`. Spark Master waits for Cassandra to be "Healthy" before starting.
- * **Idempotency**: All Spark batch jobs use `overwrite` mode to ensure re-running a job doesn't duplicate data in the Gold layer.
- * **Caching**: The `promote\_to\_gold.py` script uses `.cache()` on the aggregated DataFrame to prevent redundant computation when writing to multiple sinks (Postgres + Parquet).

Setup & Installation

1. Prerequisites

- * **Docker Desktop** (with at least 8GB RAM allocated).
- * **Python 3.10+** (for local feeders).
- * **VcXsrv or Xming** (Optional: only if you want to see the native UI on Windows).

2. First-Time Setup (The Intern's Path)

Follow these exact steps to avoid common "broken environment" issues:

- * **Clone & Open**: Open the project folder in VS Code.
- * **Verify Docker**: Open Docker Desktop and ensure it shows "Engine Running" in the bottom left.
- * **The UI Helper**: If you are on Windows, start **VcXsrv**. Choose "One large window", "Display number 0", and **crucially** check "Disable access control". This allows the Docker container to show its window on your screen.
- * **Run the script**: Open PowerShell in the project root and run:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process # Allows script to run
./Run-Pipeline.ps1
```

- * **Monitor the Web UI**: Open your browser to `http://localhost:8080`. You should see 3 Workers listed. If you see 0, wait 60 seconds; they are still waking up!

3. Troubleshooting FAQ

- * **"Connection Refused"**: Usually means a container is still starting. Cassandra can take up to 2 minutes to become "Healthy".
- * **"No module named yfinance"**: Ensure you have run `pip install -r requirements.txt` on your host machine if you are running the feeder outside of Docker.
- * **"X11 connection refused"**: Your VcXsrv is either not running or "Disable access control" wasn't

checked.

Maintenance & Operations

1. Checking Logs

If something goes wrong, the first place to look is the logs/ directory. Each major service has its own dedicated folder:

- * **Spark**: `logs/spark/`
- * **Cassandra**: `logs/cassandra/`
- * **Kafka**: `logs/kafka/`
- * **Fincept UI**: `logs/fincept/`

2. Cleaning Data

To wipe the cluster clean and start from scratch:

- * Stop the containers: `docker-compose down -v` (The `-v` flag deletes all volumes/data).
- * Clean local data folders: Delete contents of `data/processed/`.

3. Resource Management

If your computer is lagging, you can limit the Spark workers in docker-compose.yml:

- * Change `SPARK\_WORKER\_MEMORY=4G` to `2G`.
 - * Change `SPARK\_WORKER\_CORES=2` to `1`.
-

Security & Configuration

1. Default Credentials

Service	Username	Password
PostgreSQL	`admin`	`financial_secret`
Cassandra	(Default - Disabled)	(Default - Disabled)

2. Environment Variables

You can configure the pipeline by editing the .env file (if present) or directly in docker-compose.yml:

- * `FINCEPT\_KAFKA\_BROKER`: The address of the Kafka broker.
 - * `FINCEPT\_CASSANDRA\_HOST`: The primary Cassandra seed node.
-

Development Workflow: Adding New Scripts

To add a new processing script to the pipeline:

- * **Place your script** in ``scripts/processing/`` or ``scripts/ingestion/``.
- * **Import Utils**: Use ``from scripts.utils.spark_utils import create_spark_session`` to stay consistent.
- * **Submit to Cluster**: Use the following command template to run your script on the Spark cluster:

```
docker exec -i spark-master /opt/bitnami/spark/bin/spark-submit --master spark://spark-master:7077 /opt/spark
```

Project Structure

- * ``/scripts/ingestion``: Feeders and Kafka producers.
- * ``/scripts/processing``: Spark batch and streaming jobs.
- * ``/docker``: Custom Dockerfiles for specialized images.
- * ``/config``: Configuration files for Cassandra, Spark, and Kafka.
- * ``/docs``: Specialized tutorials (e.g., Ubuntu VM setup, Word Count).

Important Configuration

- * **JDBC Drivers**: The pipeline requires the PostgreSQL JDBC driver to be present in ``/opt/bitnami/spark/jars/`` inside the Spark containers.
- * **Connectivity**: Ensure your firewall allows communication on ports ``9092`` (Kafka), ``9042`` (Cassandra), and ``5432`` (Postgres).